

Challenging coding assessment – JusPay 2nd Round

1) Common Coding Themes in Juspay's Second Round

- **Graph and Tree Algorithms:** Expect problems that require a deep understanding of graph traversal and tree manipulation.
- **Concurrency and Thread Safety:** Some questions may test your knowledge of multithreading and synchronization mechanisms.
- **Real-World Scenarios:** Problems might simulate real-world applications, such as Implementing a **multi-user locking system** on an n-ary tree, Designing a **scheduling or queuing system** with fairness, Building a **data processing pipeline** that can handle concurrency.
- **Learn Basics of Concurrency** (Optional but Impressive) - Juspay deals with payment systems — concurrency and synchronization are real issues. What is a thread/process?, Basics of mutex, lock, race condition, deadlock, Writing a basic thread-safe class in Java or Python

2) Focus Coding Topics on:

- **Trees & Graphs:** DFS, BFS, Union-Find, Topological Sort.
- **Recursion & Backtracking:** Used in tree and lock problems.
- **Heaps & Priority Queues:** For problems involving optimization.
- **HashMaps & Sets:** For fast lookups in graph and string problems.
- **Greedy & Dynamic Programming:** Especially interval-based and DP-on-trees problems.

3) JusPay Coding Questions asked in Previous interviews:

1. Tree Locking System

Problem Type: Object-Oriented Design + Tree Traversal

You are given an n-ary tree. Implement:

- lock(node_id, user_id)
- unlock(node_id, user_id)
- upgrade(node_id, user_id) — allowed only if:
 - Node is not locked.
 - No ancestors are locked.
 - At least one descendant is locked.

Key skills: Tree traversal, OOP, logical constraints.

2. Largest Cycle in a Graph

Problem Type: Graph Traversal

Given a directed graph, return the length of the longest cycle.

- If no cycles exist, return -1.

Key skills: DFS, visited tracking, cycle detection.

3. Meeting Cell (Shortest Common Path)

Problem Type: Graph BFS

- Given a graph and two nodes, find the closest common node both can reach.

Key skills: Bidirectional BFS, shortest path.

4. Rate Limiter Simulator

Problem Type: System Simulation + Logic

You are designing a system that allows:

- A user to make up to N requests per T seconds.
- After exceeding the limit, the user is blocked for B seconds.
- `allow_request(user_id, timestamp)`

Key skills: Queue, sliding window, system simulation.

5. Transaction Rollback System

Problem Type: Stack Simulation

Design a system that supports:

- `begin_transaction()`
- `write(key, value)`
- `read(key)`
- `rollback()` – undoes changes in the current transaction.

Key skills: Stack, hash maps, OOP

6. Maximum Weight Node

Problem Type: Graph + Arrays

- You're given an array `edges[]` where `edges[i]` is the node that node `i` points to. Each node has a weight. Find the node where the sum of weights of its incoming edges is maximum.

Key skills: Reverse mapping, graph in-degree analysis.

7. Deadlock Detection in a System

Problem Type: Graph + DFS

You are given a list of processes and resource requests. Detect whether a deadlock occurs.

Key skills: Graph cycle detection.

8. Thread-safe Counter System

Problem Type: Concurrency Simulation

Simulate a shared counter accessed by multiple threads. Implement:

- `increment()`
- `decrement()`
- `get()`

Make sure race conditions are avoided.

Key skills: Locking simulation, mutex logic (pseudo-code is fine for freshers).

9. Access Control with Inheritance

Problem Type: Tree + Permissions

Each node is a resource. A user has access to a node if they have:

- Direct permission
- Or inherited permission from an ancestor node.

Given permission grants and queries like `can_access(user_id, node_id)`, respond accordingly.

Key skills: Tree traversal, caching, access propagation.

10. Event Scheduler

Problem Type: Interval Management

Design a scheduler that:

- Adds an event with a time interval.
- Rejects overlapping events.
- Supports add, delete, and `find_free_slots(start, end)`.

Key skills: Interval trees, sorting, binary search.

4) Preparation Tips

- **Don't Just Solve** — Analyze After you finish a problem, always discuss complexity ($O(n)$, $O(n \log n)$, etc.). Talk about alternatives or improvements.
- **Certain problem patterns reappear** - Have templates ready for: DFS/BFS on trees, Graph traversal with visited set, LRU cache / sliding window, Multi-threading with locks (pseudo-code).
- Code clean and modular - Always use classes and docstrings, Write your own test inputs/scripts and use VSCode shortcuts / snippets.

5) Resources

<https://medium.com/%40kamalchandra265/juspay-on-campus-interview-experience-for-batch-2025-34c364f5b6b8>

<https://www.geeksforgeeks.org/tag/juspay/>

<https://unstop.com/blog/juspay-coding-questions>

<https://youtu.be/dnzwqLhGncg>

https://www.youtube.com/results?search_query=juspay+interview+experience

<https://www.baeldung.com/java-concurrency>

<https://realpython.com/intro-to-python-threading/>

<https://www.youtube.com/watch?v=WldMTtUWqTg>